



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
IIC2513 - TECNOLOGÍAS Y APLICACIONES WEB

Entrega 4

Tecnologías y Aplicaciones Web

2º semestre 2026 - Profesor Hernán Cabrera

Fecha de Entrega: Domingo 08 de Noviembre a las 23:59 hrs.

Introducción

En las entregas anteriores construyeron la interfaz visual en React (E_2) e implementaron la persistencia relacional con PostgreSQL/Sequelize junto a la estructura base del servidor Koa (E_3).

El objetivo central de esta **Entrega 4** es consolidar la **lógica de negocio del juego en el servidor**, implementar un sistema completo y seguro de **autenticación y autorización mediante JWT y hashing con bcrypt**, incorporar comunicación bidireccional en tiempo real con **WebSockets**, asegurar la calidad del código mediante **pruebas unitarias con Jest** y realizar el **despliegue del Backend y Base de Datos en la nube**.

En esta etapa, el servidor Koa asume definitivamente su rol de **árbitro estricto**: es el único responsable de validar las jugadas, alternar los turnos, verificar condiciones de victoria/derrota y proteger los recursos según el rol del usuario.

Indicaciones Generales

Para esta entrega deberán avanzar en la madurez técnica de su aplicación, cumpliendo con los siguientes requerimientos principales:

- Lógica de Negocio y Servidor Árbitro:** Implementación rigurosa de las reglas del juego en el backend, gestión de turnos y validación de estados válidos sin datos hardcoded.
- Autenticación y Criptografía:** Registro e inicio de sesión funcional, almacenamiento seguro de contraseñas hasheadas con **bcrypt** (con *salt*).
- Autorización y Roles con JWT:** Emisión, firma y verificación de *JSON Web Tokens*, middlewares de protección de rutas y diferenciación entre roles (Jugador vs. Administrador).
- Esqueleto del Panel de Administración:** Creación de rutas protegidas accesibles exclusivamente por usuarios con rol de administrador.

- e. **WebSockets en Tiempo Real:** Configuración de un servidor de WebSockets para gestionar al menos un evento síncrono del juego (sala de espera, cambio de turno o actualización del tablero).
- f. **Pruebas Unitarias Automatizadas:** Al menos 5 pruebas con **Jest** sobre los controladores clave de la lógica del juego.
- g. **Despliegue del Backend en PaaS:** API y base de datos PostgreSQL desplegadas públicamente y operativas en internet (Render, Supabase o equivalente).

Requerimientos Detallados

1. Lógica del Juego y Servidor Árbitro

El servidor debe controlar la lógica de la partida y asegurar la consistencia del estado del juego:

- **Validación de Jugadas:** Cada petición de acción enviada por un jugador debe ser auditada por el backend. Si una jugada viola las reglas, se ejecuta fuera de turno o con recursos insuficientes, debe rechazarse con un código HTTP **400 Bad Request** o **422 Unprocessable Entity**.
- **Gestión de Turnos y Estados:** El servidor debe alternar automáticamente el jugador activo, procesar eventos fortuitos/aleatorios según sus reglas y evaluar condiciones de victoria o empate.
- **Persistencia Real:** Todo cambio de estado (posiciones, puntajes, inventarios) debe actualizarse directamente en PostgreSQL a través de Sequelize. Prohibido el uso de variables globales en memoria volátil o datos hardcodeados.

2. Autenticación y Criptografía (bcrypt)

- **Registro de Usuarios (*Sign Up*):** Endpoint para registrar nuevos usuarios con credenciales y asignación de rol inicial.
- **Hashing Seguro: Queda estrictamente prohibido guardar contraseñas en texto plano.** Toda contraseña debe ser hasheada utilizando **bcrypt** (con *salt rounds* apropiados) antes de guardarse en la base de datos.
- **Inicio de Sesión (*Login*):** Comparación segura del hash mediante **bcrypt.compare**. Si las credenciales son inválidas, responder con **401 Unauthorized**.

3. Autorización basada en Roles y JWT

- **Generación de Token:** Al iniciar sesión exitosamente, el backend debe emitir un token **JWT** firmado con una clave secreta (**JWT_SECRET**), incluyendo el **id** y el **rol** del usuario en el payload.
- **Middlewares de Autenticación en Koa:** Middleware que extraiga el token desde la cabecera **Authorization: Bearer <token>**, verifique su validez y monte el usuario autenticado en el contexto (**ctx.state.user**).

- **Control de Acceso (RBAC):** Rutas protegidas según el perfil del usuario. Los usuarios no autenticados no pueden realizar jugadas, y ciertas rutas de auditoría/gestión deben quedar reservadas exclusivamente para administradores.
- **Esqueleto del Panel de Administración:** Definición de al menos una ruta protegida funcional para el rol de Administrador (preparando la gestión integral de la Entrega 5).

4. WebSockets y Eventos en Tiempo Real

Se debe configurar una capa de **WebSockets** (usando librerías como `socket.io` o `ws`) integrada al servidor Koa:

- **Registro de Conexión:** Identificar y registrar la sesión o el identificador del socket al autenticarse.
- **Evento en Tiempo Real:** Implementar y documentar al menos un flujo interactivo en tiempo real entre los participantes. Ejemplos válidos:
 - Notificación en vivo de jugadores uniéndose a una sala de espera (*lobby*).
 - Actualización automática del tablero y cambio de turno sin requerir recarga manual.
 - Transmisión de mensajes de chat o eventos de victoria/derrota a los jugadores conectados.

5. Pruebas Unitarias Automatizadas (Jest)

El backend debe incluir una suite de pruebas automatizadas utilizando **Jest** (y herramientas auxiliares como `supertest`):

- Se deben implementar **al menos 5 pruebas unitarias/integración no triviales**, enfocadas en los controladores y servicios que gobiernan la lógica del juego y la autenticación.
- Deben abarcar tanto **casos de éxito** (jugada válida, login correcto) como **casos de error** (jugada fuera de turno, contraseña incorrecta, acceso a ruta protegida sin token).
- Todas las pruebas deben ejecutarse de manera automatizada mediante el comando `npm test`.

6. Despliegue del Backend y Base de Datos (PaaS)

- La API de Koa debe ser desplegada en un proveedor PaaS (recomendado: **Render** o similar).
- La base de datos PostgreSQL debe residir en una instancia en la nube (recomendado: **Supabase** o la base de datos administrada de Render).
- El backend desplegado debe responder solicitudes HTTP y WebSockets reales vía internet. El enlace a la API debe incluirse en el `README.md`.

7. Gestión del Proyecto y Documentación

- **README.md:** Instrucciones completas para ejecutar pruebas unitarias (`npm test`), variables de entorno requeridas (`.env.example`), documentación del evento WebSocket implementado y el enlace al despliegue público.

- **Tablero Kanban:** Actualizado con las tareas de la Entrega 4, mostrando responsables, fechas límite y estados de avance.

Condiciones y Restricciones

- Stack obligatorio: **Node.js, Koa, PostgreSQL, Sequelize, WebSockets y Jest.** Prohibido el uso de TypeScript.
- **Seguridad:** Prohibido el uso de contraseñas sin hashear o tokens con secretos expuestos en el código. **El archivo .env NO debe subirse al repositorio.**
- **Trazabilidad en GitHub:** Se evaluará el historial de commits y *Pull Requests* de todos los integrantes.
- **No se aceptarán entregas por correo electrónico ni fuera de plazo.**

Sobre la Calificación

La calificación evaluará la solidez del servidor como árbitro de las reglas, la correcta implementación criptográfica y de tokens JWT, la integración de WebSockets, la cobertura de testing con Jest y la disponibilidad del backend en producción.

Criterios de Evaluación

Criterio	Esencial	Intermedio	Completo	Pje.
1. Lógica del Juego y Servidor Árbitro	Reglas incompletas o en cliente	Validación básica sin persistencia	Servidor valida jugadas, gestiona turnos, victoria/derrota y persiste en PostgreSQL	15
2. Criptografía y Hashing (Bcrypt)	Contraseñas en texto plano	Hashing parcial o sin salt	Registro y login con contraseñas hasheadas y comparadas usando <code>bcrypt</code>	15
3. Autorización JWT y Roles	Token sin validar en rutas	Validación de token sin roles	Emisión de JWT, middlewares en Koa, rutas protegidas y roles (Admin/Jugador)	15
4. WebSockets en Tiempo Real	Servidor WebSocket sin eventos	Comunicación unidireccional	Canal bidireccional funcional para al menos un evento en tiempo real documentado	15
5. Pruebas Unitarias con Jest	Menos de 3 pruebas o triviales	5 pruebas solo de casos de éxito	Al menos 5 pruebas no triviales (éxito y error) en controladores ejecutables con <code>npm test</code>	10
6. Despliegue Backend en PaaS	Despliegue caído o solo local	Backend desplegado pero con BD local	API Koa y PostgreSQL desplegados en PaaS, operativos y accesibles públicamente	10
7. Esqueleto Panel de Administración	Vista o ruta sin protección	Ruta creada con permisos débiles	Ruta protegida funcional exclusiva para usuarios con rol administrador	6
8. Manejo de Errores y Seguridad	Errores 500 no capturados	Manejo de errores parcial	Códigos HTTP semánticos (400, 401, 403, 404, 422), variables seguras en <code>.env</code>	6
9. Documentación y README	README sin comandos ni link	Instrucciones incompletas de deploy	Documentación de endpoints, eventos WebSocket, testing y enlace al backend en PaaS	4
10. Kanban y Trabajo en GitHub	Tablero desactualizado	Tareas asignadas sin seguimiento	Tablero Kanban activo, commits equitativos y uso correcto de Pull Requests	4

Puntaje Total: 100 puntos

Requisitos Mínimos para Nota 4.0

Para optar a una calificación igual o superior a 4.0, el grupo debe cumplir obligatoriamente en nivel **Completo** con los siguientes ítems:

1. Lógica del juego y servidor como árbitro de reglas y turnos.
2. Almacenamiento seguro de contraseñas hasheadas con **bcrypt**.
3. Autorización mediante JWT, middlewares de autenticación y control de roles.
4. Servidor de WebSockets con al menos un evento en tiempo real operativo.
5. Al menos 5 pruebas unitarias automatizadas con Jest.
6. Despliegue funcional del Backend y Base de Datos en PaaS.

Política de Referencia y Uso de Inteligencia Artificial

En el desarrollo del proyecto grupal, los estudiantes podrán utilizar material proporcionado por el equipo docente, recursos disponibles en internet y herramientas basadas en Inteligencia Artificial (por ejemplo, ChatGPT, Claude, GitHub Copilot, entre otras). Sin embargo, todas las fuentes utilizadas deben ser debidamente referenciadas y el equipo debe demostrar un **aporte propio significativo** en la solución implementada.

Si se utiliza Inteligencia Artificial, se deberá indicar de forma explícita, transparente y detallada en una sección dedicada del archivo `README.md` de la entrega (o en un documento anexo dentro de la carpeta `/docs`), incluyendo:

- **Alcance y contexto:** Indicar con precisión qué partes del trabajo, componentes, estilos o funciones fueron apoyadas o generadas mediante IA.
- **Registro de interacción:** Adjuntar los *prompts* utilizados, los fragmentos (*snippets*) de código entregados por la herramienta y los enlaces públicos a las conversaciones o **capturas de pantalla legibles de los chats**.
- **Aporte y análisis crítico:** Explicar qué modificaciones, adaptaciones, correcciones e integraciones realizó el equipo sobre dicho contenido para incorporarlo al proyecto.

Régimen de Penalizaciones y Descuentos:

- En caso de que el aporte de los integrantes no sea considerado sustancial (por ejemplo, si la mayor parte del trabajo corresponde a código generado automáticamente sin comprensión ni personalización por parte del grupo), la nota podrá ser rebajada según el criterio del equipo docente.
- **Uso no referenciado o referenciado incorrectamente:** El uso de herramientas de IA sin su debida declaración, o con un reporte incompleto (e.g., no adjuntar capturas, prompts o no transparentar el alcance real de su uso), constituye una falta grave a esta política y se aplicará un **descuento directo de entre 0 a 100 puntos sobre la calificación final de la entrega**, dependiendo de la magnitud y gravedad de la omisión detectada. El uso no declarado de IA podrá derivar adicionalmente en una calificación final de nota 1,1, además de las medidas disciplinarias que determine el equipo docente.